


# Human-in-the-Loop
## in LangGraph

Interrupt • Approve • Resume • Safety Patterns

Detailed Lecture Notes with Full Code & Extra Concepts

*CampusX Lecture Series*

`interrupt()` `Command(resume=)` `MemorySaver` `Action Approval` `Safety`

# ■ TABLE OF CONTENTS ■


| | | |
|---|---|---|
| 01 | **Overview & The Problem** | *Why agents need human oversight* |
| 02 | **Why Use HITL?** | *Accuracy, Safety & Ethics* |
| 03 | **Common HITL Patterns** | *4 patterns every agent needs* |
| 04 | **LangGraph HITL Mechanics** | *interrupt(), Command, Checkpointer* |
| 05 | **Code: Simple Approval Bot** | *Basic HITL with interrupt* |
| 06 | **Code: Stock Purchase Bot** | *Real-world selective approval* |
| 07 | **The Interrupt Lifecycle** | *Visual step-by-step data flow* |
| 08 | **Advanced: Dynamic Routing** | *Approve, Reject, Edit, Escalate* |
| 09 | **Advanced: Error Handling** | *Timeouts, retries & fallbacks* |
| 10 | **Production Patterns** | *Best practices for real apps* |
| 11 | **Anti-Patterns to Avoid** | *Common mistakes and how to fix them* |
| 12 | **Summary & Quick Reference** | *Everything at a glance* |

# 01 Overview & The Problem

**Human-in-the-Loop (HITL)** is a critical design pattern for Agentic AI. While agents are designed for autonomy — deciding what to do and executing actions — the LLM brain powering them is **not yet perfect**. HITL places a human checkpoint at critical moments.

### ■ The Core Tension: Autonomy vs. Control

There is a fundamental **tension** in agentic AI design: you want the agent to be autonomous enough to be useful, but controlled enough to be safe. HITL is the mechanism that balances this tension by letting the agent handle routine tasks independently while requiring human sign-off for critical decisions.

## Three Problems with Unchecked Agents:

### ■ Hallucination

LLMs can confidently generate wrong information. An agent acting on a hallucinated fact could send wrong payments, book wrong flights, or delete wrong files.

### ■ Ambiguity Misinterpretation

"Schedule meeting for next Friday" — this Friday or next week? The agent guesses instead of asking. Wrong guess = wasted time for everyone.

### ■ No Accountability

If an autonomous agent transfers $50,000 to the wrong account, who is responsible? Without HITL, there's no human approval trail for audits.

## 02 Why Use HITL?

### ■ Accuracy & Error Correction

Humans catch LLM mistakes **before** they become final. Example: An invoice-reading agent misreads "$1,000" as "$12,000". With HITL, the human corrects the amount before payment is processed. Without it, $11,000 is overpaid.

### ■ Safety & Accountability

High-stakes actions like **financial transactions, file deletions, email sending, or database modifications** must have human sign-off. This creates an **audit trail** — a record that a human approved the action, critical for compliance and legal protection.

### ■ Ethical Alignment

Humans ensure output matches company values and cultural sensitivity. Example: A customer support agent drafts a response that is technically correct but lacks empathy. The human reviewer adds warmth and understanding before sending.

### ■ When to Apply HITL

**Rule of Thumb:** If an action is **irreversible** (can't be undone), **expensive** (costs money/resources), or **external** (affects people outside the system), it should have HITL approval. Read-only actions (fetching data, searching) usually don't need it.

## 03 Common HITL Patterns

There are four major HITL patterns, each suited to different scenarios. Understanding which pattern to apply is a key design skill for building production agents.

### 1. ✔ Action Approval

MOST COMMON

The agent proposes an action and **pauses for Yes/No**. Used for financial transactions, sending emails, modifying data, API calls with side effects.

■ *Example: Agent: "Pay $50 to vendor X?" → Human: "Yes" → Payment executes*

### 2. ✍ Output Review

CONTENT QUALITY

The agent generates content (blog post, email, report) and the human **reviews/edits** before publishing. The human can modify, rewrite, or approve as-is.

■ *Example: Agent drafts tweet → Human edits tone → Final version posted*

### 3. ■ Ambiguity Clarification

SMART HANDLING

When the user's request is unclear, the agent **pauses to ask clarifying questions** instead of guessing. Prevents compounding errors from wrong assumptions.

■ *Example: User: "Book flight Friday" → Agent: "This Friday or next?" → User: "Next"*

### 4. ■ Escalation

FAILSAFE

The agent recognizes it **cannot handle** the request (too complex, out of scope, or sensitive) and hands the entire conversation to a human agent. Graceful degradation.

■ *Example: User asks complex legal question → Agent: "Transferring to specialist..."*

# 04 LangGraph HITL Mechanics

LangGraph implements HITL using three components that work together: the **interrupt** function, the **Checkpointer**, and the **Command** object.

### ■ interrupt() — The Pause Button

When the graph executes `interrupt()` inside a node, three things happen:

① **PAUSE:** Execution stops immediately at that exact line

② **SAVE:** The current state is saved via the Checkpointer (like a game save)

③ **SEND:** The payload inside interrupt() is sent to the frontend/user

```
from langgraph.types import interrupt
```

### ■ Checkpointer — The Save System

**Crucial for HITL.** Without a checkpointer, the graph forgets where it stopped. The checkpointer saves the **exact execution state** — which node, which line, all variables — so it can resume hours or even days later when the human responds.

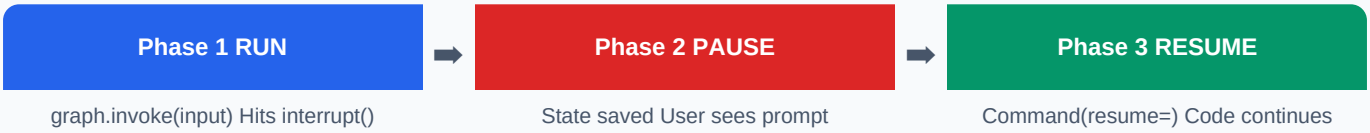
```
from langgraph.checkpoint.memory import MemorySaver
```

### ■ Command(resume=) — The Play Button

To restart the graph after a pause, you call `invoke()` again with a `Command(resume=value)` object. The `value` becomes the **return value** of the interrupt() call — the code continues exactly where it left off, with the human's input now available.

```
from langgraph.types import Command
```

## The Three-Phase HITL Cycle:



# 05 Code: Simple Approval Bot

## ■ Example 1: Basic Approval Flow

This is the simplest HITL example: **every** question sent to the LLM requires human approval first. The graph pauses, shows the question, and only calls the LLM if approved.

THE NODE WITH INTERRUPT

```python
from langgraph.types import interrupt, Command

def chat_node(state):
    # 1. PAUSE: Send question to human for approval
    decision = interrupt({
        "question": state["question"],
        "instruction": "Do you approve this?"
    })

    # 2. RESUME: Code continues here after human responds
    # 'decision' now contains the user's Command(resume=...) value
    if decision.get("approved") == "yes":
        response = llm.invoke(state["question"])
        return {"messages": [response]}
    else:
        return {"messages": ["Request denied by human."]}
```

RUNNING THE GRAPH (Client/Frontend Side)

```python
# STEP 1: First invoke — graph runs until interrupt
config = {"configurable": {"thread_id": "1"}}
inputs = {"question": "Explain Gradient Descent"}

events = graph.invoke(inputs, config=config)
# Graph PAUSES here. 'events' contains the interrupt payload.
print("Bot asking for approval...")

# ... Human reviews and decides ...
user_input = "yes"

# STEP 2: Resume with Command
from langgraph.types import Command

graph.invoke(
    Command(resume={"approved": user_input}),
    config=config  # SAME thread_id!
)
```

## ■ Common Pitfall

**Critical:** The `thread_id` must be **identical** in both the initial invoke and the resume invoke. This is how the Checkpointer knows which paused state to load. Different thread_id = different (or no) state = crash.

# 06 Code: Stock Purchase Bot

This real-world example shows **selective HITL** — only risky actions require approval. The bot has two tools: one safe (read-only) and one risky (financial action).

| Tool | Type | Risk Level | HITL? |
|---|---|---|---|
| **get_stock_price** | Read-only / Fetch data | ● Low | No |
| **purchase_stocks** | Financial transaction | ● HIGH | ✔ YES |

THE RISKY TOOL WITH INTERRUPT

```python
@tool
def purchase_stocks(ticker: str, quantity: int):
    """Buys stocks. Only use when user explicitly asks to buy."""

    # ■ HITL: Pause before executing financial transaction
    user_approval = interrupt(
        f"Approve buying {quantity} shares of {ticker}? (yes/no)"
    )

    # Resume: user_approval contains the string from Command(resume=)
    if user_approval == "yes":
        # Execute actual purchase (API call, mocked here)
        return f"Successfully purchased {quantity} shares of {ticker}."
    else:
        return "Transaction cancelled by user."
```

## Complete User Workflow:

1. **■ User: "What is Apple's stock price?"**
   Bot calls get_stock_price — no pause needed. Returns $150.
2. **■ User: "Buy 10 shares"**
   Bot calls purchase_stocks. Graph hits interrupt() and PAUSES.
3. **■ Graph PAUSED**
   User sees: "Approve buying 10 shares of AAPL? (yes/no)"
4. **✔ User: "Yes"**
   Frontend sends Command(resume="yes"). Graph RESUMES inside the tool.
5. **■ Purchase executes**
   Bot: "Successfully purchased 10 shares of AAPL." Done!

## 07 The Interrupt Lifecycle

Understanding the exact data flow during an interrupt is key to debugging HITL systems. Here is what happens at every stage:

**1** `graph.invoke(input, config)`
Normal execution begins. State is created/loaded from checkpointer.

**2** `Node reaches interrupt(payload)`
Execution FREEZES at this exact line. The interrupt payload (dict, string, etc.) is captured.

**3** `Checkpointer saves full state`
Everything is persisted: node position, local variables, message history, interrupt payload.

**4** `Control returns to caller`
graph.invoke() returns. The return value contains the interrupt payload for the frontend to display.

**5** `Frontend shows prompt to user`
The UI displays the approval request. This can take seconds, minutes, or even days.

**6** `User provides response`
Human types "yes", "no", edits content, etc. Frontend packages this as Command(resume=value).

**7** `graph.invoke(Command(resume=val), config)`
Graph resumes. Checkpointer loads the frozen state. The interrupt() call now returns val.

**8** `Execution continues from interrupt line`
Code after interrupt() runs with the human's input. Normal execution resumes to completion.

## 08 Advanced: Dynamic Routing

In production, HITL isn't just "yes/no." Humans might want to **approve, reject, edit, or escalate**. Here's how to implement multi-option routing:

MULTI-OPTION INTERRUPT HANDLER

```
def approval_node(state):
    decision = interrupt({
        "draft": state["draft_email"],
        "options": ["approve", "reject", "edit", "escalate"]
    })

    action = decision["action"]

    if action == "approve":
        return {"status": "send_email"}
    elif action == "reject":
        return {"status": "cancelled"}
    elif action == "edit":
        # Human provided edited version
        return {"draft_email": decision["edited_text"],
                "status": "review_again"}
    elif action == "escalate":
        return {"status": "transfer_to_human_agent"}
```

| Option | When to Use | Frontend Command |
|---|---|---|
| ✔ Approve | Content/action is correct as-is | `Command(resume={"action":"approve"})` |
| ✘ Reject | Cancel the entire action | `Command(resume={"action":"reject"})` |
| ✍ Edit | Modify then continue | `Command(resume={"action":"edit","edited_text":"..."})` |
| ■ Escalate | Hand off to human agent | `Command(resume={"action":"escalate"})` |

# 09 Advanced: Error Handling

Production HITL systems must handle edge cases gracefully. What if the human never responds? What if they respond with unexpected input? Here are essential patterns:

### ■ Timeout Handling

A human might forget to respond. Your system needs a timeout mechanism to prevent indefinitely paused workflows from accumulating.

- Set a **maximum wait time** (e.g., 24 hours) after which the workflow auto-cancels or auto-approves (depending on risk level).
- Send **reminder notifications** at intervals (e.g., after 1 hour, 4 hours, 12 hours).
- Track **pending interrupts** in a dashboard so managers can see stalled workflows.

### ■ Input Validation

Always validate the human's response before continuing execution:

- **Type checking:** If you expect a dict with an "approved" key, verify it exists before accessing it.
- **Value validation:** If expected values are "yes"/"no", handle unexpected inputs like "maybe" or empty strings.
- **Re-interrupt:** If input is invalid, call interrupt() again to re-ask the question.

ROBUST INTERRUPT WITH VALIDATION

```
def safe_approval(state):
    while True: # Re-ask until valid input
        decision = interrupt({
            "prompt": "Approve? (yes/no)",
            "data": state["pending_action"]
        })

        if decision in ("yes", "no"):
            break
        # Invalid input → loop re-asks

    return {"approved": decision == "yes"}
```

### ■ Audit Trail Logging

**Logging Best Practice:** Always log interrupt events with timestamps, thread IDs, the payload sent, and the human's response. This creates the audit trail that makes HITL valuable for compliance and debugging. Include: who approved, when, and what they saw.

# 10 Production Patterns

## Best Practices for Production HITL Systems:

### ■ Risk-Based Approval

Not every action needs human approval. Classify actions by risk level: **Low** (auto-approve), **Medium** (approval for first occurrence, then auto), **High** (always require approval).

### ■ Approval Dashboard

Build a centralized dashboard showing all pending approvals, who requested them, timestamps, and urgency levels. This prevents approval bottlenecks.

### ■ Role-Based Access

Different approval levels for different roles: junior staff can approve low-risk actions, managers approve medium-risk, executives approve high-risk.

### ■ Multi-Channel Notifications

Send approval requests via multiple channels (email, Slack, SMS) to ensure humans see them promptly. Include one-click approve/reject buttons.

### ■ Graceful Degradation

If the approval system is down, have a fallback: queue actions for later approval, or auto-reject with notification. Never silently auto-approve high-risk actions.

### ■ Always Use DB Checkpointer

In production, ALWAYS use PostgresSaver or similar. MemorySaver means a server restart loses ALL pending approvals. Users would have to re-submit everything.

# 11 Anti-Patterns to Avoid

Common mistakes when implementing HITL that lead to broken, insecure, or frustrating systems:

### ■ Approving Everything

If EVERY action requires approval, users get "approval fatigue" and start rubber-stamping everything without reading. This defeats the purpose of HITL entirely.

**Fix:** Use risk-based tiers. Auto-approve low-risk actions.

### ■ No Timeout on Pauses

A paused graph that waits forever accumulates stale states in the checkpointer. After days/weeks, the context may no longer be relevant.

**Fix:** Set expiry times. Cancel or re-prompt after 24-48 hours.

### ■ Using MemorySaver in Production

A server restart wipes ALL paused states. Every pending approval vanishes. Users must re-do all their work.

**Fix:** Always use PostgresSaver/RedisSaver for production.

### ■ No Context in Approval Prompt

Showing "Approve? (yes/no)" without telling the user WHAT they're approving. Humans can't make informed decisions without context.

**Fix:** Always include the full action details in the interrupt payload.

### ■ Ignoring the Edit Path

Only offering yes/no when the human might want to modify the action slightly (e.g., change amount from 10 shares to 5).

**Fix:** Support edit/modify responses alongside approve/reject.

## 12 Summary & Quick Reference

| Component | Purpose | Code |
|---|---|---|
| interrupt() | Pause graph, send data to human | `decision = interrupt(payload)` |
| Command(resume=) | Resume graph with human input | `invoke(Command(resume=val), config)` |
| Checkpointer | Save/load paused state | `compile(checkpointer=MemorySaver())` |
| thread_id | Must match between pause & resume | `{"configurable":{"thread_id":"1"}}` |
| get_state() | Check if graph is paused | `graph.get_state(config).next` |

| ✔ Action Approval | Output Review | Ambiguity Clarification | Escalation |
|---|---|---|---|
| Yes/No for risky actions | Edit content before publish | Ask before guessing | Hand off to human agent |

### The Big Takeaway

HITL is not a limitation on agents — it's a **superpower**. It lets you deploy autonomous systems with confidence by keeping humans in control of critical decisions. The pattern is simple: **interrupt()** to pause, **Checkpointer** to save, **Command(resume=)** to continue.

*Trust the agent for routine tasks. Verify for critical ones.*

**interrupt()** Pause + Prompt **+** **Checkpointer** Save State **=** **Safe Agentic AI!**